

LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 5 Mei 2026 yang membahas tentang konsep dasar *Single Linked List* serta berbagai operasi yang dapat dilakukan, seperti penambahan, pencarian, dan penghapusan data. Penulis juga memahami bagaimana setiap *node* saling terhubung melalui *pointer*. Materi ini penting karena menjadi fondasi dalam memahami struktur data.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 7 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Single Linked List	2
2.2 Kode Pemograman	2
2.2.1 Class NodeSLL	2
2.2.2 Class TambahSLL	3
2.2.3 Output TambahSLL	5
2.2.4 Class PencarianSLL	5
2.2.5 Output PencarianSLL.....	7
2.2.6 Class HapusSLL.....	7
2.2.7 Output HapusSLL	9
BAB III PENUTUP	11
3.1 Kesimpulan.....	11
DAFTAR PUSTKA.....	12
TUGAS PRAKTIKUM.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data yang dinamis membutuhkan struktur data yang fleksibel. Salah satu struktur data yang dapat digunakan adalah *Single Linked List*. Berbeda dengan *array*, *Linked List* tidak memiliki ukuran tetap dan setiap elemennya saling terhubung melalui *pointer*.

Single Linked List terdiri dari node yang memiliki dua bagian, yaitu data dan *pointer* ke node berikutnya. Struktur ini memungkinkan penambahan dan penghapusan data dilakukan dengan lebih efisien, terutama pada bagian awal dan tengah *list*.

Pada praktikum ini, dilakukan implementasi berbagai operasi pada *Single Linked List*, seperti menambah *node* di depan, belakang, dan posisi tertentu, melakukan pencarian data, serta menghapus *node*. Hal ini bertujuan untuk memahami cara kerja dan penerapan *Linked List* dalam program Java.

1.2 Tujuan Praktikum

1. Memahami konsep dasar *Single Linked List*.
2. Mengetahui struktur *node* (data dan *pointer*).
3. Mampu mengimplementasikan operasi penambahan data.
4. Mampu melakukan pencarian data dalam *list*.
5. Mampu menghapus *node* dalam berbagai kondisi.

1.3 Manfaat Praktikum

1. Memahami cara kerja struktur data *Linked List* secara dinamis.
2. Melatih penggunaan *pointer* dalam pemrograman.
3. Mengetahui proses penambahan dan penghapusan *node*.
4. Meningkatkan logika dalam pengelolaan data berantai.

BAB II

PEMBAHASAN

2.1 Single Linked List

Single Linked List merupakan salah satu struktur data yang terdiri dari kumpulan *node* yang saling terhubung secara berurutan melalui sebuah *pointer*. Setiap *node* memiliki dua bagian utama, yaitu data untuk menyimpan nilai dan *pointer* (*next*) yang menunjuk ke node berikutnya. *Node* terakhir akan memiliki *pointer* bernilai *null* yang menandakan akhir dari *list*.

Berbeda dengan *array*, *Single Linked List* tidak memiliki ukuran tetap sehingga lebih fleksibel dalam menambah dan menghapus data. Penambahan data dapat dilakukan di berbagai posisi, seperti di awal (*head*), di akhir, maupun di tengah *list*. Begitu juga dengan penghapusan data yang dapat dilakukan berdasarkan posisi tertentu tanpa harus menggeser seluruh elemen seperti pada *array*.

Dalam implementasinya di Java, *Single Linked List* dibuat menggunakan *class node* yang saling terhubung. Beberapa operasi dasar yang dilakukan pada praktikum ini antara lain menambah *node* di depan, belakang, dan posisi tertentu, melakukan pencarian data, serta menghapus *node*. Proses *traversal* juga digunakan untuk menelusuri seluruh *node* dari awal hingga akhir.

2.2 Kode Pemograman

2.2.1 Class NodeSLL

```
1 package pekan5_2511531010;
2
3 public class NodeSLL_2511531010 {
4     // Node bagian data
5     int data_1010;
6     // Pointer ke node berikutnya
7     NodeSLL_2511531010 next_1010;
8     // Konstuktur menginisialisasi node dengan data
9     public NodeSLL_2511531010(int data_1010) {
10         this.data_1010 = data_1010;
11         this.next_1010 = null;
```

```

12     }
13 }

```

Gambar 2.1

Class ini digunakan untuk merepresentasikan sebuah *node*. Di dalamnya terdapat variabel data untuk menyimpan nilai dan *next* sebagai *pointer* yang menghubungkan *node* ke *node* berikutnya. Konstruktor digunakan untuk menginisialisasi data saat *node* dibuat, dan *pointer* diatur bernilai *null* sebagai tanda belum terhubung ke *node* lain.

2.2.2 Class TambahSLL

```

1  package pekan5_2511531010;
2
3  public class TambahSLL_2511531010 {
4
5      public static NodeSLL_2511531010 insertAtFront(NodeSLL_2511531010
head_1010, int value_1010) {
6          NodeSLL_2511531010 new_node_1010 = new
NodeSLL_2511531010(value_1010);
7          new_node_1010.next_1010 = head_1010;
8          return new_node_1010;
9      }
10
11     public static NodeSLL_2511531010 insertAtEnd(NodeSLL_2511531010
head_1010, int value_1010) {
12         NodeSLL_2511531010 newNode_1010 = new
NodeSLL_2511531010(value_1010);
13         if (head_1010 == null) {
14             return newNode_1010;
15         }
16         NodeSLL_2511531010 last_1010 = head_1010;
17         while (last_1010.next_1010 != null) {
18             last_1010 = last_1010.next_1010;
19         }
20         last_1010.next_1010 = newNode_1010;
21         return head_1010;
22     }
23
24     static NodeSLL_2511531010 GetNode_1010 (int data_1010) {
25         return new NodeSLL_2511531010(data_1010);
26     }
27
28     static NodeSLL_2511531010 insertPos_1010(NodeSLL_2511531010
headNode_1010, int position_1010, int value_1010) {
29         NodeSLL_2511531010 head_1010 = headNode_1010;
30         if (position_1010 < 1)
31             System.out.print("Invalid position");
32         if (position_1010 == 1) {

```

```

33         NodeSLL_2511531010 new_node_1010 = new
NodeSLL_2511531010(value_1010);
34         new_node_1010.next_1010 = head_1010;
35         return new_node_1010;
36     } else {
37         while (position_1010-- != 0) {
38             if (position_1010 == 1) {
39                 NodeSLL_2511531010 newNode_1010 =
GetNode_1010(value_1010);
40                 newNode_1010.next_1010 =
headNode_1010.next_1010;
41                 headNode_1010.next_1010 = newNode_1010;
42                 break;
43             }
44             headNode_1010 = headNode_1010.next_1010;
45         }
46     }
47     if (position_1010 != 1)
48         System.out.print("Posisi di luar jangkauan");
49     return head_1010;
50 }
51
52 public static void printList_1010(NodeSLL_2511531010 head_1010) {
53     NodeSLL_2511531010 curr_1010 = head_1010;
54     while (curr_1010.next_1010 != null) {
55         System.out.print(curr_1010.data_1010 + " --> ");
56         curr_1010 = curr_1010.next_1010;
57     }
58     if (curr_1010.next_1010 == null) {
59         System.out.print(curr_1010.data_1010);
60     }
61     System.out.println();
62 }
63
64 public static void main(String[] args) {
65     NodeSLL_2511531010 head_1010 = new NodeSLL_2511531010(2);
66     head_1010.next_1010 = new NodeSLL_2511531010(3);
67     head_1010.next_1010.next_1010 = new NodeSLL_2511531010(5);
68     head_1010.next_1010.next_1010.next_1010 = new
NodeSLL_2511531010(6);
69
70     System.out.print("Senarai berantai awal: ");
71     printList_1010(head_1010);
72
73     System.out.print("tambah 1 simpul di depan: ");
74     int data_1010 = 1;
75     head_1010 = insertAtFront(head_1010, data_1010);
76     printList_1010(head_1010);
77
78     System.out.print("tambah 1 simpul di belakang: ");
79     int data2_1010 = 7;
80     head_1010 = insertAtEnd(head_1010, data2_1010);
81     printList_1010(head_1010);
82

```

```

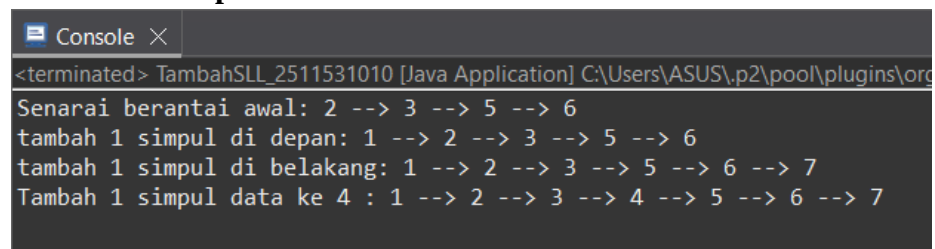
83         System.out.print("Tambah 1 simpul data ke 4 : ");
84         int data3_1010 = 4;
85         int pos_1010 = 4;
86         head_1010 = insertPos_1010(head_1010, pos_1010,
data3_1010);
87         printList_1010(head_1010);
88     }
89 }

```

Gambar 2.2

Class ini berisi *method* untuk menambahkan *node* ke dalam *Single Linked List*. Terdapat *method insertAtFront* untuk menambah *node* di depan, *insertAtEnd* untuk menambah di belakang, dan *insertPos* untuk menambah pada posisi tertentu. Selain itu, terdapat *method printList* untuk menampilkan isi *list*. Pada bagian *main*, program membuat *linked list* awal, kemudian melakukan beberapa operasi penambahan *node*.

2.2.3 Output TambahSLL



```

<terminated> TambahSLL_2511531010 [Java Application] C:\Users\ASUS\p2\pool\plugins\org
Senarai berantai awal: 2 --> 3 --> 5 --> 6
tambah 1 simpul di depan: 1 --> 2 --> 3 --> 5 --> 6
tambah 1 simpul di belakang: 1 --> 2 --> 3 --> 5 --> 6 --> 7
Tambah 1 simpul data ke 4 : 1 --> 2 --> 3 --> 4 --> 5 --> 6 --> 7

```

Gambar 2.3

Output menampilkan kondisi awal *linked list*, kemudian menampilkan perubahan setelah dilakukan penambahan *node* di depan, belakang, dan posisi tertentu. Setiap operasi akan menghasilkan tampilan *list* yang berbeda sesuai dengan perubahan data yang dilakukan.

2.2.4 Class PencarianSLL

```

1 package pekan5_2511531010;
2
3 public class PencarianSLL_2511531010 {
4     static boolean searchKey_1010(NodeSLL_2511531010 head_1010, int
key_1010) {
5         NodeSLL_2511531010 curr_1010 = head_1010;
6         while (curr_1010 != null) {
7             if (curr_1010.data_1010 == key_1010)
8                 return true;

```



```

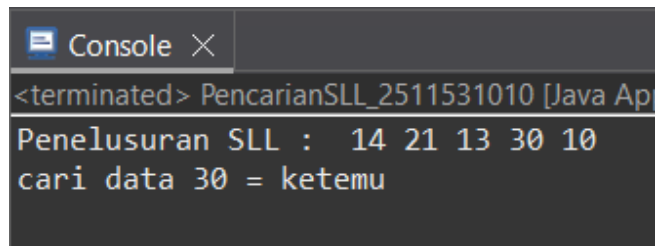
9         curr_1010 = curr_1010.next_1010;
10    }
11    return false;
12 }
13
14 public static void traversal_1010(NodeSLL_2511531010 head_1010)
15 {
16     // mulai dari head
17     NodeSLL_2511531010 curr_1010 = head_1010;
18     // telusuri sampai pointer null
19     while (curr_1010 != null) {
20         System.out.print(" " + curr_1010.data_1010);
21         curr_1010 = curr_1010.next_1010;
22     }
23     System.out.println();
24 }
25
26 public static void main(String[] args) {
27     NodeSLL_2511531010 head_1010 = new NodeSLL_2511531010(14);
28     head_1010.next_1010 = new NodeSLL_2511531010(21);
29     head_1010.next_1010.next_1010 = new NodeSLL_2511531010(13);
30     head_1010.next_1010.next_1010.next_1010 = new
NodeSLL_2511531010(30);
31     head_1010.next_1010.next_1010.next_1010.next_1010 = new
NodeSLL_2511531010(10);
32
33     System.out.print("Penelusuran SLL : ");
34     traversal_1010(head_1010);
35
36     // data yang akan dicari
37     int key_1010 = 30;
38     System.out.print("cari data " + key_1010 + " = ");
39     if (searchKey_1010(head_1010, key_1010))
40         System.out.println("ketemu");
41     else
42         System.out.println("tidak ada");
43 }
44 }

```

Gambar 2.4

Class ini digunakan untuk melakukan pencarian data dalam *linked list*. Method *searchKey* akan menelusuri setiap *node* hingga menemukan data yang dicari atau mencapai akhir *list*. Selain itu, terdapat *method traversal* untuk menampilkan seluruh isi *linked list*.

2.2.5 Output PencarianSLL



```
Console X
<terminated> PencarianSLL_2511531010 [Java Ap
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

Gambar 2.5

Output menampilkan seluruh isi *linked list* terlebih dahulu, kemudian menampilkan hasil pencarian data. Jika data ditemukan, akan muncul keterangan “ketemu”, dan jika tidak ditemukan akan muncul “tidak ada”.

2.2.6 Class HapusSLL

```
1 package pekan5_2511531010;
2
3 public class HapusSLL_2511531010 {
4     // Fungsi untuk menghapus head
5     public static NodeSLL_2511531010
6     deleteHead_1010(NodeSLL_2511531010 head_1010) {
7         // jika SLL kosong
8         if (head_1010 == null) {
9             return null;
10        }
11        // Pindahkan head ke node berikutnya
12        head_1010 = head_1010.next_1010;
13        return head_1010;
14    }
15    // Fungsi menghapus node terakhir SLL
16    public static NodeSLL_2511531010
17    removeLastNode_1010(NodeSLL_2511531010 head_1010) {
18        // jika list kosong, return null
19        if (head_1010 == null) {
20            return null;
21        }
22        // jika list satu node, hapus node dan return null
23        if (head_1010.next_1010 == null) {
24            return null;
25        }
26        // Temukan node kedua terakhir
27        NodeSLL_2511531010 secondLast_1010 = head_1010;
28        while (secondLast_1010.next_1010.next_1010 != null) {
29            secondLast_1010 = secondLast_1010.next_1010;
30        }
31        // hapus node terakhir
32        secondLast_1010.next_1010 = null;
33        return head_1010;
34    }
35 }
```

```

33 // Fungsi menghapus node di posisi tertentu
34 public static NodeSLL_2511531010
deleteNode_1010(NodeSLL_2511531010 head_1010, int position_1010) {
35     NodeSLL_2511531010 temp_1010 = head_1010;
36     NodeSLL_2511531010 prev_1010 = null;
37     // jika linked list null
38     if (temp_1010 == null) {
39         return head_1010;
40     }
41     // Kasus 1: Menghapus head (posisi 1)
42     if (position_1010 == 1) {
43         head_1010 = temp_1010.next_1010;
44         return head_1010;
45     }
46     // Kasus 2: menghapus node di tengah
47     // Telusuri ke node yang akan dihapus
48     for (int i_1010 = 1; temp_1010 != null && i_1010 <
position_1010; i_1010++) {
49         prev_1010 = temp_1010;
50         temp_1010 = temp_1010.next_1010;
51     }
52     // Jika node ditemukan, hapus node
53     if (temp_1010 != null) {
54         prev_1010.next_1010 = temp_1010.next_1010;
55     } else {
56         System.out.println("Data pada posisi tersebut tidak
ada");
57     }
58     return head_1010;
59 }
60 // Fungsi mencetak SLL
61 public static void printList_1010(NodeSLL_2511531010 head_1010)
{
62     if (head_1010 == null) {
63         System.out.println("List Kosong");
64         return;
65     }
66     NodeSLL_2511531010 curr_1010 = head_1010;
67     while (curr_1010 != null) {
68         System.out.print(curr_1010.data_1010);
69         if (curr_1010.next_1010 != null) {
70             System.out.print(" --> ");
71         }
72         curr_1010 = curr_1010.next_1010;
73     }
74     System.out.println();
75 }
76 // Kelas main
77 public static void main(String[] args) {
78     // Buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
79     NodeSLL_2511531010 head_1010 = new NodeSLL_2511531010(1);
80     head_1010.next_1010 = new NodeSLL_2511531010(2);
81     head_1010.next_1010.next_1010 = new NodeSLL_2511531010(3);

```

```

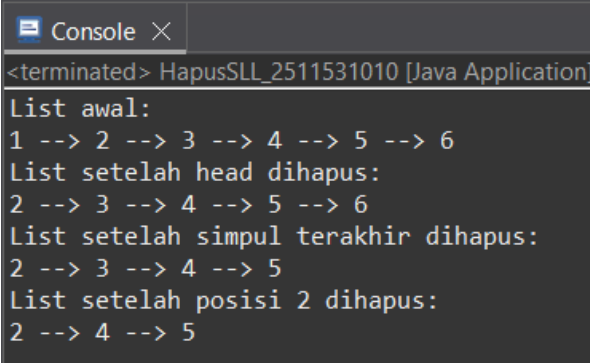
82     head_1010.next_1010.next_1010.next_1010 = new
NodeSLL_2511531010(4);
83     head_1010.next_1010.next_1010.next_1010 = new
NodeSLL_2511531010(5);
84     head_1010.next_1010.next_1010.next_1010.next_1010
= new NodeSLL_2511531010(6);
85     // cetak list awal
86     System.out.println("List awal: ");
87     printList_1010(head_1010);
88     // Hapus head
89     head_1010 = deleteHead_1010(head_1010);
90     System.out.println("List setelah head dihapus: ");
91     printList_1010(head_1010);
92     // Hapus node terakhir
93     head_1010 = removeLastNode_1010(head_1010);
94     System.out.println("List setelah simpul terakhir dihapus:
");
95     printList_1010(head_1010);
96     // Hapus node at position 2
97     int position_1010 = 2;
98     head_1010 = deleteNode_1010(head_1010, position_1010);
99     System.out.println("List setelah posisi " + position_1010 +
" dihapus: ");
100    printList_1010(head_1010);
101    }
102 }

```

Gambar 2.6

Class ini berisi *method* untuk menghapus *node* dalam *linked list*. Terdapat beberapa *method* seperti *deleteHead* untuk menghapus *node* pertama, *removeLastNode* untuk menghapus *node* terakhir, dan *deleteNode* untuk menghapus *node* pada posisi tertentu. Selain itu, terdapat *method* untuk mencetak isi *list* sebelum dan sesudah penghapusan.

2.2.7 Output HapusSLL



```

Console X
<terminated> HapusSLL_2511531010 [Java Application]
List awal:
1 --> 2 --> 3 --> 4 --> 5 --> 6
List setelah head dihapus:
2 --> 3 --> 4 --> 5 --> 6
List setelah simpul terakhir dihapus:
2 --> 3 --> 4 --> 5
List setelah posisi 2 dihapus:
2 --> 4 --> 5

```

Gambar 2.7

Output menampilkan kondisi awal *linked list*, kemudian menampilkan perubahan setelah dilakukan penghapusan *node* di bagian awal, akhir, dan posisi tertentu. Setiap langkah menunjukkan bagaimana data dalam *list* berkurang dan berubah sesuai operasi yang dilakukan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Single Linked List* merupakan salah satu struktur data yang sangat penting dalam pemrograman karena mampu mengelola data secara dinamis tanpa batasan ukuran seperti pada *array*. Setiap data disimpan dalam bentuk *node* yang saling terhubung melalui *pointer*, sehingga memungkinkan proses penambahan dan penghapusan data dilakukan dengan lebih fleksibel.

Melalui berbagai implementasi yang telah dibuat, seperti penambahan *node* di depan, belakang, dan posisi tertentu, pencarian data, serta penghapusan *node*, dapat dipahami bahwa *Single Linked List* memiliki peran penting dalam pengelolaan data yang sering berubah. Selain itu, proses *traversal* juga membantu dalam menelusuri seluruh isi data secara berurutan dari awal hingga akhir.

Praktikum ini juga menekankan pentingnya pemahaman konsep *pointer* dalam menghubungkan setiap *node*, karena kesalahan dalam pengelolaan *pointer* dapat menyebabkan data tidak terhubung dengan baik. Dengan memahami konsep ini, mahasiswa dapat lebih mudah mengembangkan program yang lebih kompleks.

DAFTAR PUSTKA

- [1] Oracle Corporation. 2023. *Java Documentation*. Diakses dari <https://docs.oracle.com/javase/8/docs/>

TUGAS PRAKTIKUM

Soal:

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. Aturan Penamaan (Wajib Dipatuhi)

1. Nama Kelas : Gunakan NIM lengkap.
Contoh : Pasien_2411531005 dan RumahSakit_2411531005.
2. Nama Variabel dan Method : Setiap variabel dan method wajib menggunakan 4 digit terakhir NIM.
Contoh : String namaPasien_1005, String penyakit_1005.
3. Pointer next : Variabel pointer ke node berikutnya juga wajib diberi akhiran 4 digit NIM.
Contoh : Pasien_2411531005 next_1005.

3. Struktur Program

1. Kelas ADT Pasien (Pasien_NIMLengkap)

Atribut :

- Nama Pasien
- Keluhan / Penyakit
- Nomor Antrian (int)
- Pointer next ke node Pasien berikutnya

Method :

- Constructor untuk menginisialisasi semua atribut.
- Selektor (Getter) untuk setiap atribut.

- Mutator (Setter) untuk setiap atribut termasuk pointer next.
2. Kelas Driver (RumahSakit_NIMLengkap)
- Daftarkan Pasien (Insert at Tail) : Menambahkan node pasien baru di akhir linked list. Jika list kosong, node baru langsung menjadi head. Gunakan sebuah variabel counter untuk menyimpan nomor antrian terakhir. Setiap kali fungsi Daftarkan Pasien dipanggil, nomor antrian akan bertambah secara otomatis (*auto-increment*).
 - Panggil Pasien (Delete Head) : Menghapus node terdepan dan menampilkan data pasien yang dipanggil. Head digeser ke node berikutnya.
 - Tampilkan Antrian (Display) : Menelusuri linked list dari head hingga null dan menampilkan semua data pasien beserta posisinya.
 - Cari Pasien (Search) : Pencarian dilakukan berdasarkan nama pasien secara Case-Insensitive (tidak membedakan huruf besar dan kecil). Tampilkan pesan jika tidak ditemukan.
 - Cek Status Antrian : Menampilkan jumlah total pasien dan informasi pasien terdepan. Tampilkan pesan jika list kosong.

Kode Pemograman:

1. Class Pasien_2511531010

```

2. package pekan5_2511531010;
3.
4. public class Pasien_2511531010 {
5.     String namaPasien_1010;
6.     String penyakit_1010;
7.     int nomorAntrian_1010;
8.     Pasien_2511531010 next_1010;
9.     // Constructor
10.    public Pasien_2511531010(String namaPasien_1010, String
        penyakit_1010, int nomorAntrian_1010) {
11.        this.namaPasien_1010 = namaPasien_1010;
12.        this.penakit_1010 = penyakit_1010;
13.        this.nomorAntrian_1010 = nomorAntrian_1010;
14.        this.next_1010 = null;
15.    }
16.    // Getter
17.    public String getNamaPasien_1010() {
18.        return namaPasien_1010;
19.    }
20.    public String getPenyakit_1010() {
21.        return penyakit_1010;
22.    }
23.    public int getNomorAntrian_1010() {
24.        return nomorAntrian_1010;
25.    }

```

```

26.     public Pasien_2511531010 getNext_1010() {
27.         return next_1010;
28.     }
29.     // Setter
30.     public void setNext_1010(Pasien_2511531010 next_1010) {
31.         this.next_1010 = next_1010;
32.     }
33. }

```

2. Class RumahSakit_2511531010

```

3. package pekan5_2511531010;
4.
5. import java.util.Scanner;
6.
7. public class RumahSakit_2511531010 {
8.
9.     static Pasien_2511531010 head_1010 = null;
10.    static int counter_1010 = 0;
11.
12.    // INSERT (Daftar Pasien)
13.    static void insert_1010(String nama_1010, String
penyakit_1010) {
14.        counter_1010++;
15.        Pasien_2511531010 newNode_1010 = new
Pasien_2511531010(nama_1010, penyakit_1010, counter_1010);
16.
17.        if (head_1010 == null) {
18.            head_1010 = newNode_1010;
19.        } else {
20.            Pasien_2511531010 temp_1010 = head_1010;
21.            while (temp_1010.next_1010 != null) {
22.                temp_1010 = temp_1010.next_1010;
23.            }
24.            temp_1010.next_1010 = newNode_1010;
25.        }
26.
27.        System.out.println("Pasien berhasil didaftarkan! Nomor
Antrian: " + counter_1010);
28.    }
29.
30.    // DELETE HEAD (Panggil Pasien)
31.    static void deleteHead_1010() {
32.        if (head_1010 == null) {
33.            System.out.println("Antrian kosong!");
34.            return;
35.        }
36.
37.        System.out.println("Memanggil pasien:");
38.        System.out.println(head_1010.namaPasien_1010 + " (" +
head_1010.penyakit_1010 + ")");
39.
40.        head_1010 = head_1010.next_1010;
41.    }

```

```

42.
43.     // DISPLAY
44.     static void display_1010() {
45.         if (head_1010 == null) {
46.             System.out.println("Antrian kosong!");
47.             return;
48.         }
49.
50.         Pasien_2511531010 temp_1010 = head_1010;
51.         while (temp_1010 != null) {
52.             System.out.println("No: " +
temp_1010.nomorAntrian_1010 +
53.                 " | Nama: " + temp_1010.namaPasien_1010 +
54.                 " | Keluhan: " + temp_1010.penyakit_1010);
55.             temp_1010 = temp_1010.next_1010;
56.         }
57.     }
58.
59.     // SEARCH (Case Insensitive)
60.     static void search_1010(String nama_1010) {
61.         Pasien_2511531010 temp_1010 = head_1010;
62.         boolean found_1010 = false;
63.
64.         while (temp_1010 != null) {
65.             if
66.             (temp_1010.namaPasien_1010.equalsIgnoreCase(nama_1010)) {
67.                 System.out.println("Pasien ditemukan!");
68.                 System.out.println("No Antrian: " +
temp_1010.nomorAntrian_1010);
69.                 found_1010 = true;
70.                 break;
71.             }
72.             temp_1010 = temp_1010.next_1010;
73.         }
74.         if (!found_1010) {
75.             System.out.println("Pasien tidak ditemukan!");
76.         }
77.     }
78.
79.     // STATUS
80.     static void status_1010() {
81.         if (head_1010 == null) {
82.             System.out.println("Antrian kosong!");
83.             return;
84.         }
85.
86.         int jumlah_1010 = 0;
87.         Pasien_2511531010 temp_1010 = head_1010;
88.
89.         while (temp_1010 != null) {
90.             jumlah_1010++;
91.             temp_1010 = temp_1010.next_1010;
92.         }

```

```

93.
94.         System.out.println("Jumlah pasien: " + jumlah_1010);
95.         System.out.println("Pasien terdepan: " +
head_1010.namaPasien_1010);
96.     }
97.
98.     // MAIN MENU
99.     public static void main(String[] args) {
100.         Scanner input_1010 = new Scanner(System.in);
101.         int pilihan_1010;
102.
103.         do {
104.             System.out.println("=== Antrian Rumah Sakit
NIM: 2511531010 ===");
105.             System.out.println("1. Daftarkan Pasien");
106.             System.out.println("2. Panggil Pasien");
107.             System.out.println("3. Tampilkan Antrian");
108.             System.out.println("4. Cari Pasien");
109.             System.out.println("5. Cek Status Antrian");
110.             System.out.println("6. Keluar");
111.             System.out.print("Pilihan: ");
112.
113.             pilihan_1010 = input_1010.nextInt();
114.             input_1010.nextLine();
115.
116.             switch (pilihan_1010) {
117.                 case 1:
118.                     System.out.print("Masukkan Nama Pasien:
");
119.                     String nama_1010 =
input_1010.nextLine();
120.                     System.out.print("Masukkan Keluhan: ");
121.                     String penyakit_1010 =
input_1010.nextLine();
122.                     insert_1010(nama_1010, penyakit_1010);
123.                     break;
124.
125.                 case 2:
126.                     deleteHead_1010();
127.                     break;
128.
129.                 case 3:
130.                     display_1010();
131.                     break;
132.
133.                 case 4:
134.                     System.out.print("Cari nama: ");
135.                     String cari_1010 =
input_1010.nextLine();
136.                     search_1010(cari_1010);
137.                     break;
138.
139.                 case 5:
140.                     status_1010();

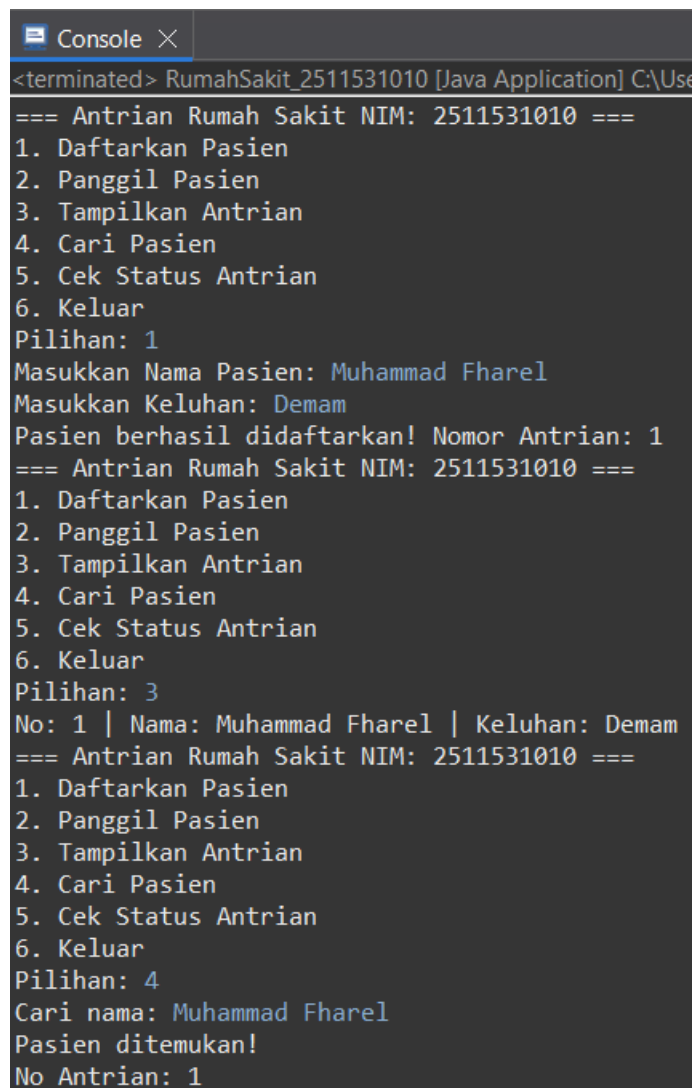
```

```

141.                 break;
142.
143.                 case 6:
144.                     System.out.println("Keluar...");
145.                     break;
146.
147.                 default:
148.                     System.out.println("Pilihan tidak
valid!");
149.             }
150.
151.         } while (pilihan_1010 != 6);
152.     }
153. }

```

3. Output



```

<terminated> RumahSakit_2511531010 [Java Application] C:\Use
=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Muhammad Fharel
Masukkan Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
No: 1 | Nama: Muhammad Fharel | Keluhan: Demam
=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Cari nama: Muhammad Fharel
Pasien ditemukan!
No Antrian: 1

```

```

=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 1
Pasien terdepan: Muhammad Fharel
=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil pasien:
Muhammad Fharel (Demam)
=== Antrian Rumah Sakit NIM: 2511531010 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Keluar...

```

Tugas ini bertujuan untuk mengimplementasikan *Single Linked List* (SLL) dalam bentuk simulasi antrian rumah sakit. Pada struktur ini, setiap data pasien disimpan dalam sebuah node yang saling terhubung melalui *pointer next*, sehingga membentuk rantai data yang dinamis.

Setiap *node* (*class* Pasien_2511531010) memiliki atribut berupa nama pasien, keluhan, nomor antrian, dan *pointer* ke *node* berikutnya. Nomor antrian dibuat otomatis menggunakan variabel *counter* yang akan bertambah setiap kali pasien baru didaftarkan.

Konsep utama yang digunakan adalah FIFO (*First In First Out*), di mana pasien yang pertama masuk akan menjadi pasien pertama yang dipanggil. Hal ini diimplementasikan dengan:

- *Insert* di akhir (*tail*) saat mendaftarkan pasien.
- *Delete* di awal (*head*) saat memanggil pasien.

Program juga menyediakan beberapa operasi penting, yaitu:

- *Insert* untuk menambah pasien ke antrian.
- *Delete Head* untuk memanggil pasien terdepan.
- *Display* untuk menampilkan seluruh antrian.
- *Search* untuk mencari pasien berdasarkan nama (tanpa membedakan huruf besar/kecil).
- *Status* untuk mengetahui jumlah pasien dan siapa yang berada di posisi terdepan.